ACM/IDS 104 APPLIED LINEAR ALGEBRA
PROBLEM SET 4: SOLUTIONS

KONSTANTIN M. ZUEV

Problem 1. (10 points) Least Squares Solution
Find the least squares solution to the system $Ax = b$, where

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 0 & -2 & 3 \\ 1 & 5 & -1 \\ -3 & 1 & 1 \end{bmatrix} \quad \text{and} \quad b = \begin{bmatrix} 0 \\ 5 \\ 6 \\ 8 \end{bmatrix} \tag{1}$$

What is the least squares error?

Solution: As we know (from Lecture 7), the least squares solutions are the solutions of the system of normal equations, $A^T Ax = A^T b$. In our case,

$$A^T A = \begin{bmatrix} 11 & 4 & -5 \\ 4 & 34 & -12 \\ -5 & -12 & 12 \end{bmatrix} \quad \text{and} \quad A^T b = \begin{bmatrix} -18 \\ 28 \\ 17 \end{bmatrix}. \tag{2}$$

The least square solution is then

$$x^* = \quad (A^T A)^{-1} A^T b \quad = \begin{bmatrix} -1 \\ 2 \\ 3 \end{bmatrix} \tag{3}$$

The least square error is

$$\text{LSE}^2 = ||b||^2 - b^T A(A^T A)^{-1} A^T b = 0 \tag{4}$$

It is easy to check that x^* is in fact the exact solution, $Ax^* = b$, and thus zero error makes perfect sense.

Problem 2. (10 points) Interpolation for Integration
Most numerical methods for evaluating integrals are based on interpolation. To compute $\int_a^b f(x)dx$, one chooses $n+1$ interpolation points $a \le x_0 < x_1 < \ldots < x_n \le b$ and replaces the integrand $f(x)$ by its interpolating polynomial $p_n(x)$ of degree n, leading to the approximation:

$$\int_a^b f(x)dx \approx \int_a^b p_n(x)dx, \tag{5}$$

where the polynomial integral can be done explicitly.

(a) (3 points) Find approximation (5) for $x_0 = a$ and $x_1 = b$ (trapezoid rule).
(b) (3 points) Find approximation (5) for $x_0 = a + \frac{1}{3}(b-a)$ and $x_1 = a + \frac{2}{3}(b-a)$ (open rule).
(c) (4 points) Test the trapezoid and open rules for accuracy on the following integrals:

$$\int_0^1 e^x dx \quad \text{and} \quad \int_0^\pi \sin x dx. \tag{6}$$

Solution:

E-mail address: kostia@caltech.edu.

(a) The interpolating polynomial (passing through $(a, f(a))$ and $(b, f(b))$) is

$$p(x) = \frac{x-b}{a-b}f(a) + \frac{x-a}{b-a}f(b) \tag{7}$$

This leads to the following approximation:

$$\begin{aligned}\int_a^b f(x)dx \approx \int_a^b p(x)dx &= \int_a^b \frac{x-b}{a-b}f(a) + \frac{x-a}{b-a}f(b)dx \\ &= (b-a)\frac{f(a)+f(b)}{2}.\end{aligned} \tag{8}$$

(b) The interpolating polynomial is

$$p(x) = \frac{x-x_1}{x_0-x_1}f(x_0) + \frac{x-x_0}{x_1-x_0}f(x_1) = \frac{f(x_1)-f(x_0)}{x_1-x_0}x + \frac{x_1f(x_0)-x_0f(x_1)}{x_1-x_0}. \tag{9}$$

The approximation is then

$$\begin{aligned}\int_a^b f(x)dx \approx \int_a^b p(x)dx &= \frac{f(x_1)-f(x_0)}{x_1-x_0}\frac{b^2-a^2}{2} + \frac{x_1f(x_0)-x_0f(x_1)}{x_1-x_0}(b-a) \\ &= \frac{3}{2}(a+b)(f(x_1)-f(x_0)) + 3(x_1f(x_0)-x_0f(x_1)).\end{aligned} \tag{10}$$

(c) For the first integral:

$$\int_0^1 e^x dx = e - 1 \approx 1.718. \tag{11}$$

Trapezoid rule:

$$\int_0^1 e^x dx \approx \frac{(e+1)}{2} \approx 1.859. \tag{12}$$

Open rule:

$$\int_0^1 e^x dx \approx \frac{3}{2}(e^{2/3} - e^{1/3}) + 2e^{1/3} - e^{2/3} = \frac{e^{1/3}+e^{2/3}}{2} \approx 1.672. \tag{13}$$

For the second integral:

$$\int_0^\pi \sin x dx = 2. \tag{14}$$

Trapezoid rule:

$$\int_0^\pi \sin x dx \approx \pi\frac{\sin 0 + \sin \pi}{2} = 0.$$

Open rule:

$$\begin{aligned}\int_0^\pi \sin x dx &\approx \frac{3\pi}{2}(\sin(2\pi/3) - \sin(\pi/3)) + \pi(2\sin(\pi/3) - \sin(2\pi/3)) \\ &= \pi\frac{\sin \pi/3 + \sin 2\pi/3}{2} = \frac{\sqrt{3}\pi}{2} \approx 2.72.\end{aligned} \tag{15}$$

Problem 3.(10 POINTS) APPLICATION OF LEAST SQUARES TO DATA FITTING

The dataset `carbig`, a built-in MATLAB dataset[1] contains various characteristics for $m = 406$ automobiles from the 1970's and 1980's. Let x_1, x_2, and y be the `Weight`, `Horsepower`, and `MPG` (miles per gallon), respectively. Let's assume the following theoretical model for the data:

$$y = f(x_1, x_2) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2. \tag{16}$$

Let $\beta^* = (\beta_0^*, \beta_1^*, \beta_2^*, \beta_3^*)^T$ denote the best fit to the data, i.e. the vector that minimizes the Euclidean norm of the residual vector $r = (r_1, \ldots, r_m)^T$, where

$$r_i = y^{(i)} - f(x_1^{(i)}, x_2^{(i)}). \tag{17}$$

[1]Can be loaded by `load carbig`.

(a) (5 points) Derive the system of normal equations on β^*.
(b) (5 points) Complete Problem 3 in `PS4.mlx`.

Solution:
(a) The linear system we are trying solve is

$$\underbrace{\begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(m)} \end{bmatrix}}_{y} = \underbrace{\begin{bmatrix} 1 & x_1^{(1)} & x_2^{(1)} & x_1^{(1)}x_2^{(1)} \\ \vdots & \vdots & \vdots & \vdots \\ 1 & x_1^{(m)} & x_2^{(m)} & x_1^{(m)}x_2^{(m)} \end{bmatrix}}_{A} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \\ \beta_3 \end{bmatrix} \tag{18}$$

The matrix A is a very tall matrix and the system is overdetermined. As a result, the exact solution does not exist. The least squares solution β^* is the solution of the normal equations:

$$A^T A\beta^* = A^T y. \tag{19}$$

For part (b), see `PS4solutions.mlx` (uploaded to the Piazza page).

Problem 4.(10 points) Polynomial Interpolation
Complete Problem 4 in `PS4.mlx`.

Solution: See `PS4solutions.mlx` (uploaded to the Piazza page).

Problem 5. (10 points) Stability of the Gram-Schmidt Algorithm
Complete Problem 5 in `PS4.mlx`.

Solution: See `PS4solutions.mlx` (uploaded to the Piazza page).

Acknowledgements. A huge thanks to Pavlos Stavrinides for editing my clumsy MATLAB m-files and converting them into neat live scripts!

ACM/IDS 104 - Problem Set 4 - MATLAB Solutions

Before writing your MATLAB code, it is always good practice to get rid of any leftover variables and figures from previous scripts.

```
clc; clear; close all;
```

Problem 3 (10 points) Application of Least Squares to Data Fitting

The dataset `carbig`, a built-in MATLAB dataset, contains various characteristics for $m = 406$ automobiles from the 1970s and 1980s. Let x_1, x_2 and *y* be the `Weight`, `Horsepower` and `MPG (Miles Per Gallon)` respectively. Let us assume the following theoretical model for the data:

$$y = f(x_1, x_2) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_1 x_2 \quad (\star)$$

Let $\beta^* = \left(\beta_0^*, \beta_1^*, \beta_2^*, \beta_3^*\right)^T$ denote the best fit to the data, i.e. the vector that minimizes the Euclidean norm of the residual vector $r = (r_1, \cdots, r_m)^T$, where:

$$r_i = y^{(i)} - f\left(x_1^{(i)}, x_2^{(i)}\right)$$

Part (a) (5 points)

Derive the system of normal equations on β^*. Do this in your written-up solutions.

Part (b) (5 points)

Find β^* by solving the normal system numerically and plot the scatter plot of the data $\left\{\left(x_1^{(i)}, x_2^{(i)}, y^{(i)}\right)\right\}, i = 1 \cdots m$, together with the fitted surface $y = f(x_1, x_2)$ given by $(\star)$.

To start, let us load the dataset, define our variables, and perform some necessary cleanup as there exist `NaN` values. We do this for you :)

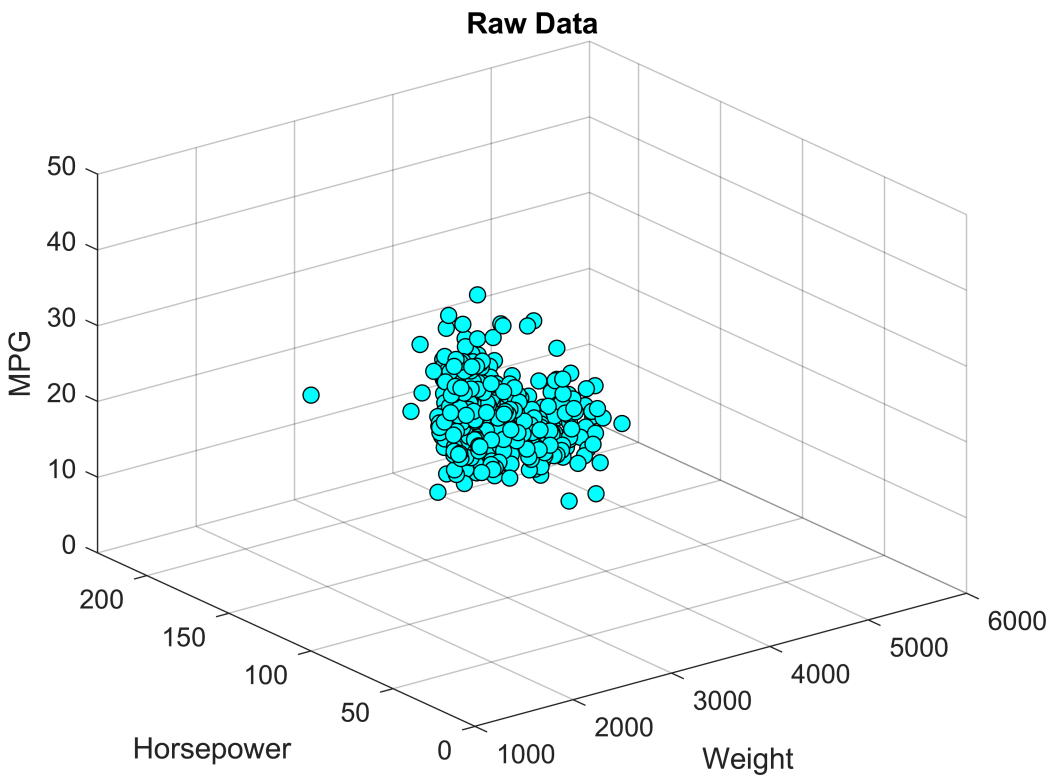
```
load carbig;
x1=Weight;
x2=Horsepower;
y=MPG;
clearvars -except x1 x2 y;
% Data cleaning
y=y(x1>0);
x2=x2(x1>0);
x1=x1(x1>0);

y=y(x2>0);
x1=x1(x2>0);
x2=x2(x2>0);

x1=x1(y>0);
x2=x2(y>0);
y=y(y>0);
```

Now, the dataset is ready for you to solve the normal system. Let us vizualize it. Drag the 3D plot around to get a better sense of the data.

```
figure;
scatter3(x1,x2,y,'MarkerEdgeColor','k','MarkerFaceColor','c');
xlabel('Weight');
ylabel('Horsepower');
zlabel('MPG');
title('Raw Data');
```

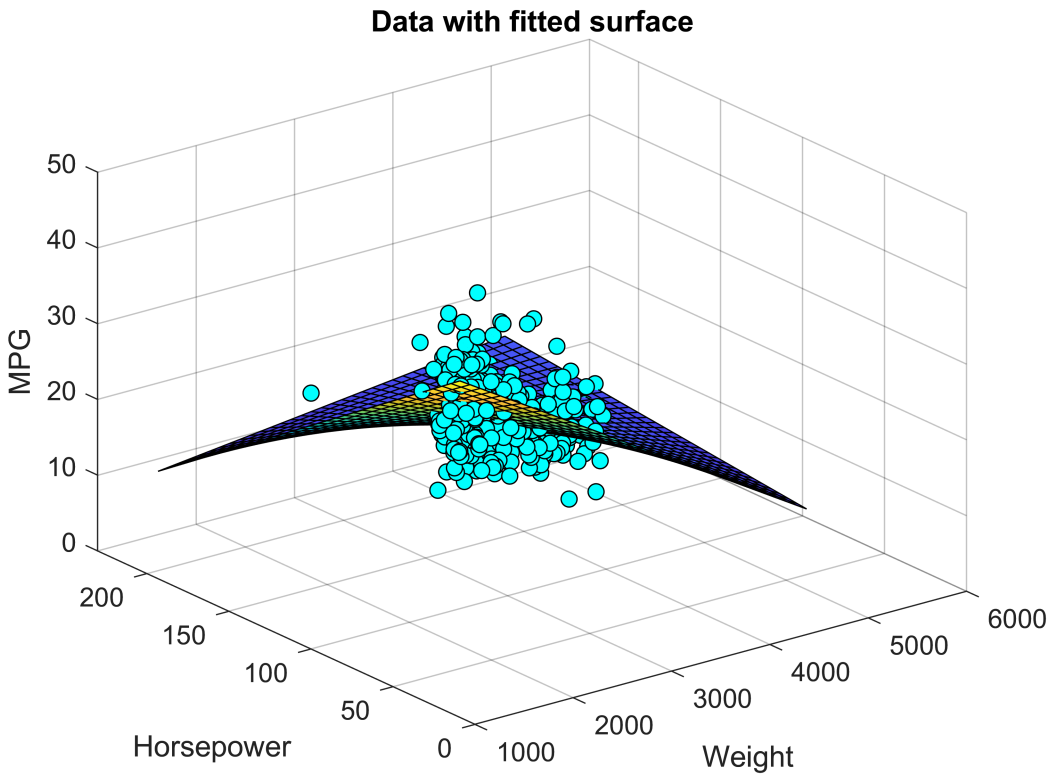


Finally, let us find the best surface to fit the data. Remember, in MATLAB, when solving a system, backslash is your best friend. Use `scatter3()` and `fsurf()` when plotting.

```
%{
Build the matrix A
Solve for beta
%}
m = length(y);
A = [ones(m,1), x1, x2, x1.*x2];
beta=A\y;
%{
this gives the least squares solution
equivalently:  beta=(A'*A)\(A'*y);
We can also use a built-in function: beta = regress(y,A);
%}

%{
```

```
Plotting
%}
figure;
scatter3(x1,x2,y,'MarkerEdgeColor','k','MarkerFaceColor','c');
hold on
f=@(x,y) beta(1)+beta(2)*x+beta(3)*y+beta(4)*x.*y;
fsurf(f,[min(x1), max(x1), min(x2), max(x2)]);
xlabel('Weight');
ylabel('Horsepower');
zlabel('MPG');
title('Data with fitted surface');
```



Problem 4 (10 points) Polynomial Interpolation

Interpolating polynomials $p(x)$ are used for approximation of complex functions $g(x)$. Intuitively, the larger the degree of the interpolating polynomial, the more accurate the approximation $g(x) \approx p(x), x \in [a, b]$. Generally, this is not true (as you will see in this problem). High degree interpolating polynomials often behave badly, especially near the ends of the interval $[a, b]$. In practice, piecewise cubic splines are often used instead of high degree polynomials.

Suppose we want to approximate the function:

$$g(x) = \frac{\cos x}{\cosh x}, \; x \in [-a, a]$$

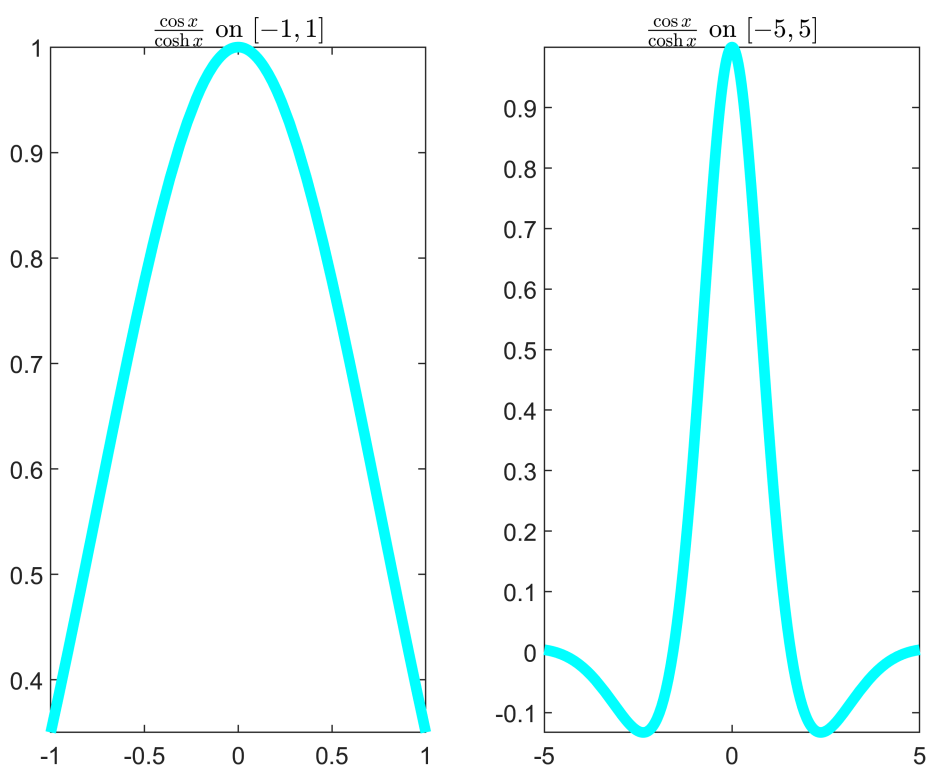
using *n* points equally spaced between $-a$ and *a*. Find the interpolating polynomials and plot them versus the function $g(x)$ for $a = 1, 5$ and $n = 3, 5, 10, 15$. In your plot, show also the data points used for interpolation. The code should produce a single figure with 8 subplots.

Useful MATLAB functions for this problem:

`linspace(), fplot(), polyval(), polyfit()`

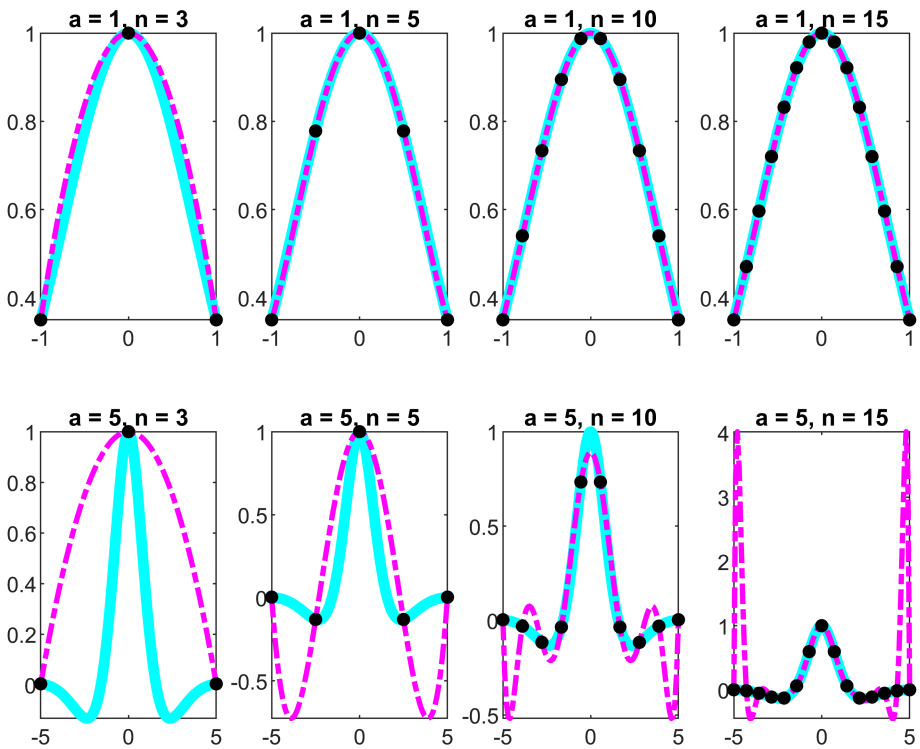
```
%{
Let us start by doing some setup
%}
g = @(x) cos(x) ./ cosh(x); % you can define functions like this
a = [1, 5]; % setting interval values
n = [3, 5, 10, 15]; % setting the number of points
sub = 1; % setting the subplot index

%{
Let us see how g(x) looks like on both intervals
%}
figure;
subplot(1, 2, 1);
fplot(g, [-a(1), a(1)], "-c", "lineWidth", 4);
title("$\frac{\cos{x}}{\cosh{x}}$ on $[-1, 1]$","Interpreter","latex");
hold on
subplot(1, 2, 2);
fplot(g, [-a(2), a(2)], "-c", "lineWidth", 4);
title("$\frac{\cos{x}}{\cosh{x}}$ on $[-5, 5]$","Interpreter","latex");
```



```
%{
Complete the nested for-loops
%}
figure;
for ival = a
    for degree = n-1
        %{
        Select degree+1 points in the interval
        Evaluate g(x) on these points
        Find the polynomial coefficients
        %}
        pts = ones(degree+1, 2); % initializing the points
        pts(:, 1) = linspace(-ival, ival, degree+1); % setting the x-values
        for i = 1 : degree+1
            pts(i, 2) = g(pts(i, 1)); % evaluating cos(x) / cosh(x)
        end
        coeffs = polyfit(pts(:, 1), pts(:, 2), degree); % coefficients
        %{
        PLOTTING
        Plot g(x), the sampled points and interpolating polynomial
        Please use different colors and linestyles
        %}
        subplot(2, 4, sub);
        fplot(g, [-ival, ival], "-c", "lineWidth", 4);
        hold on
        interpoints = linspace(-ival, ival);
        p = polyval(coeffs, interpoints); % evaluating coeffs in interval
        plot(interpoints, p, "-.m", "lineWidth", 2);
        plot(pts(:, 1), pts(:, 2), "ok", "MarkerSize", 2, "lineWidth", 3);
        title(strcat(strcat("a = ", int2str(ival)),strcat( ", n = ", int2str(degree+1))));
        sub = sub + 1; % increase subplot index
    end
end
```

```
Warning: Polynomial is badly conditioned. Add points with distinct X values, reduce the degree of the
 polynomial, or try centering and scaling as described in HELP POLYFIT.
```



Problem 5 (10 points) Stability of the Gram-Schmidt Algorithm

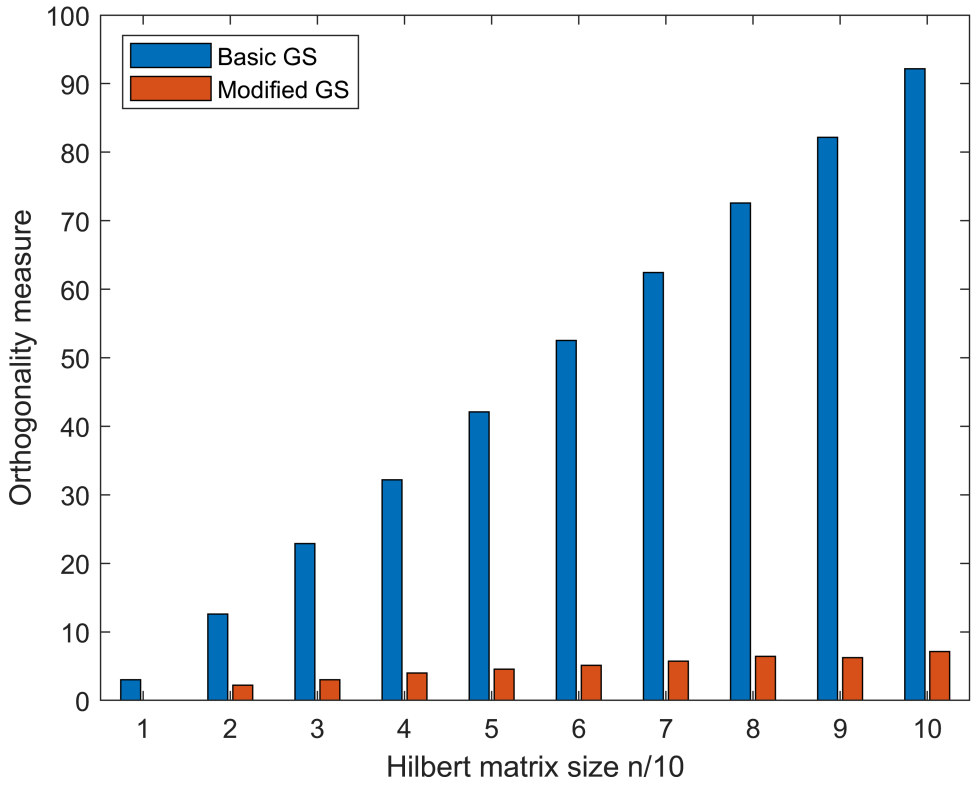
The classical Gram-Schmidt algorithm is numerically unstable. This means that, when implemented on a computer, the round-off errors can cause the output vectors to be significantly non-orthogonal. To explore the issue, perform the following computations for each $n = 10, 20, 30, \cdots, 100$:

1. Create the Hilbert matrix H_n of size *n* (using `hilb(n)`) and consider the columns $h_1, \cdots, h_n$ as a basis of $\mathbb{R}^n$. The matrix H_n is non-singular, and thus its columns indeed form a basis, but it is very close to singular (i.e. its columns are close to being linearly dependent), and this leads to numerical problems.
2. Implement the basic Gram-Schmidt algorithm to construct an orthogonal basis $\{v_1, \cdots, v_n\}$ of $\mathbb{R}^n$ from $h_1, \cdots, h_n$. Please don't use any advanced built-in function for orthogonalization (such as `orthog()`) just basic matrix operations. At the end of the process normalize your vectors so that the basis is orthonormal.
3. If vectors $v_1, \cdots, v_n$ obtained in (2) are orthonormal, then $V = [v_1, \cdots, v_n]$ must be orthogonal. As a measure of orthogonality, compute the infinite norm $\delta_V(n) = ||I_n - V^T V||_\infty$, which is a matrix norm (use `norm(A, Inf)`). The closer $\delta_V(n)$ is to zero, the closer the columns of *V* are to being orthogonal.
4. Repeat (2) and (3) to construct an orthonormal basis $\{u_1, \cdots, u_n\}$ using the modified Gram-Schmidt algorithm (which is numerically more stable) described in lecture 9 (page 46), and compute $\delta_U(n)$.
5. To compare the basic and modified Gram-Schmidt algorithms, plot $\delta_V(n)$ and $\delta_U(n)$ versus *n*.

```
%{
```

```
Complete the following for-loop following (1)-(4)
%}
for n=10:10:100
    H=hilb(n);  % Hilbert matrix, close to singular
                % Columns are close to being lineraly dependent
    %H=rand(n); % Random vectors
    % Basic Gram-Schmidt Algorithm
    V=zeros(n); %preallocation
    V(:,1)=H(:,1);
    for k=2:n
        alpha=(H(:,k)'*V(:,1:k-1))./sum(V(:,1:k-1).^2);
        V(:,k)=H(:,k)-V(:,1:k-1)*alpha';
    end
    D=diag(1./sqrt(sum(V.^2)));  % diagonal matrix, d_ii=(norm of V(:,i))^(-1)
    V=V*D; % makes the othogoanl basis orthonormal
    deltaV(n/10)=norm(eye(n)-V'*V, Inf);  % measure of orthogonality of V

    % Modified (more numerically stable) Gram-Schmidt Algorithm
    U=zeros(n); %preallocation
    U(:,1)=H(:,1)/norm(H(:,1));
    for k=2:n
        D=diag(H(:,k:n)'*U(:,k-1)); % diagonal matrix of inner products
        H(:,k:n)=H(:,k:n)- repmat(U(:,k-1),1,n-k+1)*D;
        U(:,k)=H(:,k)/norm(H(:,k));
    end
    deltaU(n/10)=norm(eye(n)-U'*U, Inf); % measure of orthogonality of U
end
%{
Visualize the comparison as specified in (5)
%}
figure;
bar([deltaV',deltaU']);
legend('Basic GS','Modified GS','Location','northwest');
xlabel('Hilbert matrix size n/10');
ylabel('Orthogonality measure');
```


Basic GS
Modified GS
Orthogonality measure
Hilbert matrix size n/10
0
10
20
30
40
50
60
70
80
90
100
1
2
3
4
5
6
7
8
9
10